

Projektuppgift

Programmering i TypeScript

Projektuppgift
Kurskatalog

Frida Pihlström



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Frida Pihlström, frpi2500@student.miun.se

Utbildningsprogram: Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: VT, 2026

Sammanfattning

I det här projektet har jag byggt en webbaserad kurskatalog ämnad för ett fiktivt lärosäte, där studenter kan söka bland alla kurser som erbjuds och sammanställa ett skräddarsytt ramschema. Projektet är utvecklat med det komponentbaserade ramverket Angular och programmeringsspråket är TypeScript.

Applikationen använder sig av en modern, reaktiv arkitektur som genom Angular Signals uppnår en hög prestanda vid sökning, sortering och filtrering. Det ger också realtidsuträkning av högskolepoängen av kurserna som läggs till i ramschemat.

Innehållsförteckning

	Sammanfattning	2
1	Inledning	Error! Bookmark not defined.
	Terminologi	4
1	Introduktion	5
1.1	Bakgrund och problemmotivering	5
1.2	Avgränsningar	5
1.3	Detaljerad problemformulering	5
2	Teori	7
2.1	Komponentbaserad arkitektur och routing	7
2.2	Tillståndshantering via Signals	7
2.3	Datastrukturer i SCSS	7
3	Metod	9
4	Konstruktion / Lösningalternativ	10
5	Resultat	14
6	Slutsatser / Analys / Diskussion	16
	Källförteckning	18

Terminologi

En del av de tekniker, begrepp och förkortningar som förekommer i rapporten.

- **JSON:** JavaScript Object Notation. Ett textbaserat datautbytesformat uppbyggt av nycklar och värden. Används för att skicka data mellan servrar och webbapplikationer.
- **SPA:** Single Page Application. En webbapplikation som bara laddar en enda HTML-sida och sen dynamiskt uppdaterar sidan i takt med att användaren interagerar med appen. Detta eliminerar behovet av sidomladdningar.
- **Standalone Component:** En komponent i Angular som hanterar sina egna import/exports utan att behöva deklarerats i en central modul.
- **Signal:** En reaktiv databehållare, som håller ett värde och automatiskt notifierar gränssnittet eller andra funktioner i samma sekund som värdet ändras.
- **Computed Signal:** En härledd signal vars värde beräknas dynamiskt baserat på en eller flera andra signaler. Värdet cachas och beräknas enbart om när ursprungssignalen ändras.
- **Effect:** En reaktiv operation som körs automatiskt i bakgrunden varje gång de signaler som läses inuti ändrar sitt tillstånd.
- **Dependency Injection:** Ett designmönster där en komponent efterfrågar externa resurser från Angulars system, istället för att skapa instansen själv.

1 Introduktion

1.1 *Bakgrund och problemmotivering*

Att navigera genom universitetsstudier och planera sin utbildning kan ofta kännas oöverskådligt. Det finns nästan oändligt med kurser, och det är svårt att hålla koll på vilka kurser man vill ta samt vilka poäng de ger och hur man ska få ihop det till 180 om man vill kunna ta ut en kandidatexamen. I det här projektet har uppdraget varit att försöka underlätta för studenter att reda ut kaoset lite grann.

Det fiktiva lärosätet har försett oss med en stor datafil i JSON-format som innehåller över 4000 unika kurser, och själva arbetet som ska utföras i det här projektet handlar om att skapa ett modernt, inbjudande och lättanvänt gränssnitt för att kunna presentera dessa kurser på ett överskådligt sätt. Vi ska skapa en Single Page Application (SPA) med ramverket Angular och TypeScript där studenten kan utforska utbudet och bygga sitt eget ramschema.

1.2 *Avgränsningar*

För att hålla projektet inom rimliga proportioner har några avgränsningar gjorts. Fokuset ligger på klientsidan, vilket innebär att applikationen inte kommunicerar med en databas på en extern server. Vi hanterar ingen inloggning, utan informationen lagras bara lokalt via localStorage.

1.3 *Detaljerad problemformulering*

För att projektet ska kunna färdigställas har jag delat upp arbetet i ett antal konkreta problem, eller delmoment, som måste lösas under arbetet.

1. Hantera prestandan vid sökning. Eftersom listan innehåller över 4000 kurser blir sidan långsam om vi ritar upp allt samtidigt. Därför måste jag bygga upp ett system där applikationen börjar tom och först när användaren skriver i sökfältet, eller väljer ämnesområde ska listan börja fyllas med kurser.
2. Filtreringssystem: Studenten ska kunna söka genom att antingen skriva in en kurskod, eller ett kursnamn men det ska också gå att se alla kurser inom ett valt ämnesområde i en dropdown-meny.
3. Skydda mot trasiga data och dubletter: JSON-filen innehåller dubletter (samma kurs flera gånger). Vi måste programmera appen så att den inte kraschar när den ritar upp dessa och vi behöver även en säkerhet så att studenten inte råkar lägga till exakt samma kurs flera gånger i sitt ramschema.

4. Länka till kursplaner: Varje kurskort måste innehålla en länk som öppnar kursplanen för den kursen i en ny flik, så att studenten kan läsa mer om kursens innehåll innan de bestämmer sig för att söka den.
5. Poängräkning och minne: Varje gång en kurs läggs till eller tas bort måste appen automatiskt räkna ut den nya totalsumman för antalet högskolepoäng. Dessutom ska ramschemat sparas i webbläsarens minne, så att studenten kan stänga ner fönstret, öppna sidan nästa dag och se sitt sparade schema.
6. Visuell färgkodning: För att göra sidan mer lättnavigerad vill jag färgkoda kurserna, genom att varje ämnesområde (som Datateknik, Omvårdnad eller Nationalekonomi) får sin egen färg.
7. Tvådelat gränssnitt (Split-screen): Jag vill ha en layout som visar kurskatalogen och det egna ramschemat sida vid sida. Detta gör att studenterna direkt kan se sitt schema växa fram på skärmen, istället för att behöva byta till en annan undersida.

2 Teori

2.1 Komponentbaserad arkitektur och routing

Angular är uppbyggt kring en komponentbaserad arkitektur, vilket innebär att gränssnittet mindre, isolerade och återanvändbara byggblock. En komponent består i grunden av tre delar: en TypeScript-klass som hanterar datalogi och tillstånd, en HTML-mall som styr strukturen, samt en SCSS-fil för lokal design.

I Angular 19 är alla komponenter som standard ”standalone”. Detta innebär att de deklarerar sina egna beroenden, till exempel om de behöver formulär, direkt i sin `@Component`-decorator via `imports: [...]` [1].

För att navigera mellan applikationens olika vyer utan att behöva ladda om webbläsaren används Angulars Router. Routern lyssnar på förändringar i URL-adressen och byter ut den komponent som visas i applikationens centrala ankare vilket är `router-outlet`-taggen `<router-outlet />` i huvudmallen.

2.2 Tillståndshantering via Signals

I webbramverk är hanteringen av tillstånd (state management) och förändringar i gränssnittet en central del. I Angular 19 används Angular Signals för att skapa en reaktiv och högpresterande applikation. En signal fungerar som en databehållare för applikationens värden.

Det unika med en signal är att den automatiskt håller reda på vart i DOM som värdet visas. När värdet inuti en signal uppdateras, vet Angular vilka specifika delar som behöver ritas om, och uppdaterar enbart dessa [2]. Detta skiljer sig från traditionella, äldre metoder i webbapplikationer där hela komponentträd behöver skannas av vid varje asynkron händelse, till exempel ett klick eller HTTP-anrop, vilket kan dra ner prestandan.

Genom att använda en så kallad computed signal går det att skapa härledda värden baserat på andra signaler. En computed signal övervakar sina underliggande signaler, beräknar sitt nya värde och cachar sedan resultatet i minnet. Vid filtrering av exempelvis en kurslista innebär det att beräkningen bara behöver köras när användaren faktiskt ändrar sin sökning.

2.3 Datastrukturer i SCSS

När man bygger upp större webbapplikationer blir ofta CSS-filerna otroligt långa och svåra att hålla koll på. Med SCSS kan man dela upp filerna för att få det mer överskådligt. Eftersom jag ville ha färgkodning där varje ämnesområde har en egen färg valde jag att använda SCSS. I SCSS finns ett verktyg som kallas för map, en map är en lista med nyckel och värde, och med det kunde jag koppla ihop de olika ämneskoderna med specifika färgnyanser [3].

Detta kombineras med en `@each`-loop där loopen tittar på listan och skapar

Projektuppgift – Kurskatalog

Frida Pihlström

2026-05-27

färdiga CSS-klasser för varje ämne på egen hand och jag slipper skriva hundratal rader med kod.

3 Metod

För att säkerställa att applikationen blir robust, prestandastark och uppfyller samtliga krav kommer arbetet att bedrivas enligt några viktiga grundprinciper och utvecklingsmetoder. Målet är att skriva en kod som är lätt att förstå, som inte kraschar och som är enkel att använda. Därför kommer följande metoder att tillämpas:

- Tydlig ansvarsuppdelning: För att koden inte ska bli en rörig massa kommer applikationen delas upp i logiska bitar. Komponenterna (HTML och SCSS) kommer tilldelas ett enda jobb: att visa upp saker på skärmen. Allt tungt arbete, som att hämta JSON-filen, filtrera kurser, hålla koll på ramschemat, kommer läggas i separata, fristående tjänster (Services). Detta kommer att underlätta framtida felsökning då det blir tydligt i vilken fil ett eventuellt fel ska spåras.
- Signals: Istället för att använda gamla metoder, där webbläsaren måste ligga och chansa om något har ändrats, kommer applikationen byggas runt Angular Signals. Genom att låta gränssnittet reagera direkt och enbart på signaler kommer applikationen att vara förberedd för modern, asynkron hantering. Detta kommer leda till att sidan fungerar snabbare vid hantering av stora datamängder.
- Defensiv kodning: Eftersom indatan i JSON-filen innehåller några felaktigheter, i form av dubletter kommer projektet att ha ett "defensivt" programmeringssätt. Det innebär att spärrar kommer att konstrueras för att hantera felaktigheter i förväg. Istället för att lita blint på att varje kurs har ett unikt ID kommer webbläsarens eget loop-index att användas för att rita upp kurserna i vyn. Dessutom kommer logiska kontroller (.some()) användas i källkoden för att stoppa felaktiga dubletter innan de får sparas i ramschemat.
- Sida-vid-sida-layout: När det kom till designen och layouten för sidan kommer användarupplevelsen stå i centrum. Istället för att studenterna ska hoppa fram och tillbaka mellan en sida för att söka och lägga till kurser och en annan sida för att se ramschemat, kommer allt visas på en sida. Ramschemat visas sida vid sida med kurserna på större skärmar, och på mindre kommer ramschemat ligga på botten av sidan, nedanför kurserna. Så fort användaren klickar på plusknappen kommer kursen att läggas till samtidigt som högskolepoängen räknas upp i realtid.

4 Konstruktion

I det här avsnittet går jag igenom hur jag faktiskt har byggt webbapplikationen i praktiken. Jag förklarar hur jag strukturerade projektet och hur jag byggde upp koden i TypeScript, HTML och SCSS för att lösa problemen som listades i inledningen.

4.1 Projektstruktur och datamodellering

Jag började arbetet med att sätta upp applikationen med hjälp av Angular. För att hålla sidan så snabb som möjligt valde jag att använda fristående komponenter (Standalone Components) direkt från start. Datafilen "miun_courses.json" lade jag i mappen "public/" så att appen kunde hämta informationen lokalt.

För att vara säker på att min TypeScript-kod inte skulle strula och att datafälten skulle bli rätt, skapade jag först av allt ett gränssnitt (interface) i filen "src/app/models/course.model.ts". Den mallen skrev jag för att matcha den exakta strukturen i lärosätets JSON-fil (se figur 4.1).

```
export interface Course {  
  courseCode: string;  
  subjectCode: string;  
  level: string;  
  progression: string;  
  courseName: string;  
  points: number;  
  institutionCode: string;  
  subject: string;  
  syllabus: string;  
}
```

Figur 4.1

4.2 Skapandet av Services

För att hålla koden ren och uppfylla kravet på ansvarsuppdelning skapade jag två tjänster (Services). Den första döpte jag till "CourseService". Dess enda uppgift är att använda Angulars "HttpClient" för att hämta JSON-filen direkt när appen startar, och sen spara ner den datan i en signal. För att komponenterna i sin tur ska kunna komma åt denna, och den andra tjänsten, använde jag mig av designmönstret Dependency Injection [6]. Det innebär att komponenterna bara efterfrågar tjänsterna via Angulars inbyggda "inject()"-funktion, istället för att skapa egna instanser av dem.

Den andra tjänsten, "ScheduleService", fick ta hand om allt som har med studentens ramschema att göra, som att lägga till kurser, ta bort och spara. För att lösa problemet med poängräkningen och minnet valde jag att använda Angular Signals tillsammans med webbläsarens localStorage [5]. Jag ställde in signalen för valda kurser så att den direkt försöker läsa från webbläsarens minne det första den gör vid uppstart (se figur 4.2).

```
export class ScheduleService {  
  // privat signal, håller koll på valda kurser  
  private selectedCoursesSignal = signal<Course[]>(  
    JSON.parse(localStorage.getItem('my_ramschema') || '[]')  
  );  
  
  // publik signal  
  public selectedCourses = this.selectedCoursesSignal.asReadonly();  
}
```

Figur 4.2

För att appen ska kunna räkna ut poängen automatiskt, utan att det drar ner prestandan, skapade jag en computed-signal. Den ligger i bakgrunden och lägger ihop poängen för alla kurser som läggs till i ramschemat och körs bara när schemat faktiskt förändras (se figur 4.3).

```
// räkna ut totala poängen automatiskt  
public totalPoints = computed(() => {  
  return this.selectedCoursesSignal().reduce((sum, course) => sum + course.points, 0);  
});  
  
// konstruktor som sparar till localStorage
```

Figur 4.3

Själva sparandet löste jag ganska smidigt med en "effect()" i tjänstens constructor. Den effekten ligger och bevakar ramschemats signal. Varje gång jag lägger till eller plockar bort en kurs, så vaknar effekten och skriver ner den nya listan direkt till localStorage (se figur 4.4).

```
constructor() {  
  effect(() => {  
    localStorage.setItem('my_ramschema', JSON.stringify(this.selectedCoursesSignal()));  
  })  
}
```

Figur 4.4

Ett av mina mål var att programmera mot dubbletter. Eftersom jag i gränssnittet hade byggt in en funktion som låser tilläggsknappen när man har valt en kurs, kan användaren i praktiken inte klicka två gånger. Men för att vara på den säkra sidan lade jag till JavaScript-funktionen ".some()" i min "addCourse"-funktion. Den fungerar som validering som dubbelkollar att kursen verkligen inte finns innan den sparas (se figur 4.5) [4].

```
// funktion för att lägga till kurs i ramschema
addCourse(course: Course): void {
  // kolla om kursen redan finns
  const alreadyExists = this.selectedCoursesSignal().some(c => c.courseCode === course.courseCode);

  if(!alreadyExists) {
    // skapa en ny array med kursen och uppdatera
    this.selectedCoursesSignal.set([...this.selectedCoursesSignal(), course]);
  }
}
```

Figur 4.5

4.3 Filtreringssystemet och sökfunktionen

För att bygga upp sökningen och filtreringen jobbade jag i komponenten "CourseCatalog". Jag skapade två signaler som håller koll på vad användaren söker i sökfältet och vad de väljer i dropdown-menyn.

Ett av mina problem i början var att det kändes överdrivet att rendera och rita upp alla 4000 kurser direkt vid start. Därför byggde jag in en spärr i min "computed"-signal (som sköter filtreringen). Jag lade in ett villkor som säger att om sökfältet är tomt, och inget är valt i dropdown-menyn, så ska funktionen bara returnera en tom array. Då renderas ingen onödig information vid sidladdning, vilket sparar mycket på webbläsarens prestanda. (se figur 4.6).

Så fort man börjar skriva något klickar filtreringen igång och letar i realtid efter matchningar, både i kursnamnet, kurskoden och ämnet.

```
// filtrerad lista
public filteredCourses = computed(() => {
  const search = this.searchQuery().toLowerCase();
  const subject = this.selectedSubject();

  // visa inget om sök och ämne är tomt
  if (search === '' && subject === '') {
    return [];
  }
});
```

Figur 4.6

4.4 HTML-vyn och smarta loopar

När jag funderade över hur designen för gränssnittet skulle se ut försökte jag komma på ett sätt att undvika tabeller eftersom de ser lite tråkiga ut, och inte fungerar så bra på mobiler. För att göra informationen lättare att ta till sig valde jag att bygga upp sidan med ett så kallat kort-gränssnitt (Card-based UI). Varje enskild kurs presenteras med ett eget "kurskort" där all relevant information är samlad på en liten yta.

Dessa kurskort strukturerades sedan upp i ett "split-screen"-gränssnitt, så att man har kurslistan på en sida och det byggda schemat bredvid.

Eftersom JSON-filen innehöll dubletter, vilket fick Angular att krascha när den försökte loopa ut korten i början, fick jag skriva om loopen. Jag använde

Angulars ”@for”-syntax och så att den ska spåra varje kort med hjälp av dess plats i listan (”track \$index”) istället för kurskoden. Då försvann kompileringsfelen.

Ett UX-tillägg jag gjorde var att lägga till en ”plus-knapp”. För att ge användaren en tydlig bekräftelse på att kursen är tillagd i schemat, blir knappen grön och får en bock istället för ett plus. Knappen blir också ”oklickbar” vilket hindrar dubbelklick och känns renare än att en ”Error”-ruta ska dyka upp (se figur 4.7).

```
<div class="card-footer">
  <span class="points-badge">{{ course.points }} HP</span>
  <a [attr.href]="course.syllabus" target="_blank" class="syllabus-link">
    Kursplan</a>

    @if (scheduleService.selectedCourses().some(c => c.courseCode === course.courseCode)) {
      <button class="add-btn already-added" disabled></button>
    } @else {
      <button class="add-btn" (click)="addCoursesToSchedule(course)">+</button>
    }
  </div>
```

Figur 4.7

4.5 Visuell färgkodning och SCSS

För att designen skulle bli mer överskådlig och inte bara en ”tabell i rutnmönster” ville jag färgkoda kurskortet efter ämnesområde. Eftersom det fanns väldigt många olika ämnen, kändes det till en början omöjligt. Sen hittade jag en lösning i att använda en separat SCSS-fil, ”_subject-colors.scss” och gjorde där en SCSS-map. Där delade jag upp lärosätets olika ämnen i huvudkategorier, och valde en färg för varje kategori och varje område fick en egen nyans av den färgen.

I min huvud-SCSS hämtade jag sedan in färgkartan med kommandot ”@use” och skrev en ”@each”-loop. Den loopen fungerar lite som en fabrik, när jag bygger appen snurrar loopen igenom listan och lägger till färgen för ämnet automatiskt. Om en kurs skulle råka tillhöra något som jag inte har lagt in i färgkartan har jag en neutral grå fallback-färg, så att designen aldrig går sönder (se figur 4.8).

```
.card-header {
  height: 25px;
  border-bottom: 2px solid #1a1a1a;
  display: flex;
  align-items: center;
  padding: 0 0.5rem;
  background-color: #7f8c8d;

  // färg baserat på ämneskod
  @each $code, $color in colors.$subject-palette {
    &.#{$code} {
      background-color: $color;
    }
  }
}
```

Figur 4.8

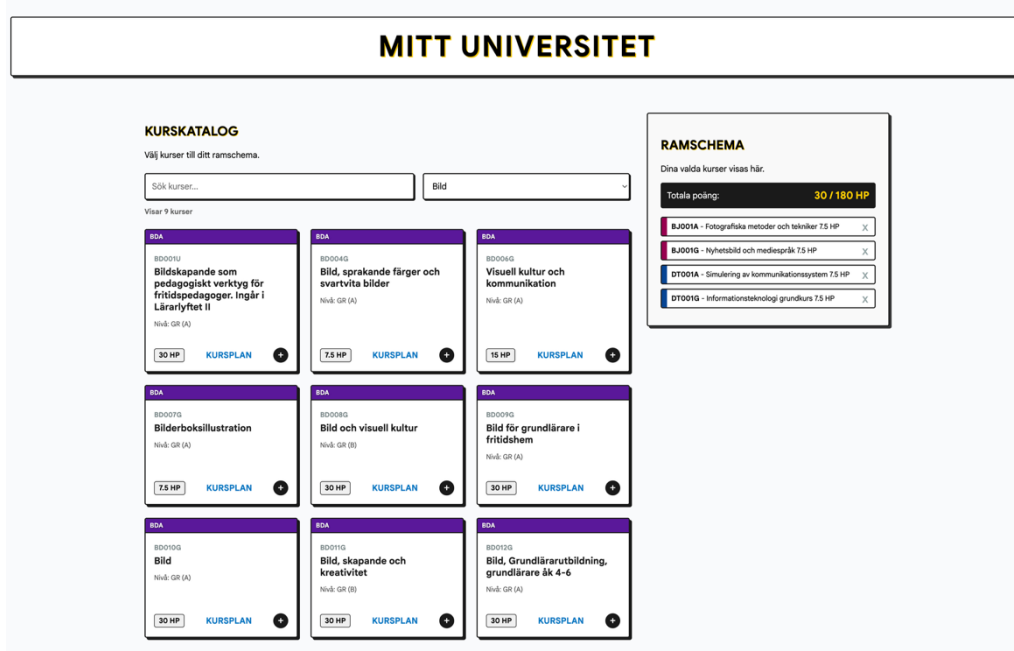
5 Resultat

I detta kapitel presenteras den färdiga webbapplikationen. Resultatet visar hur de uppsatta målen från problemformuleringen (kapitel 1.3) har uppfyllts.

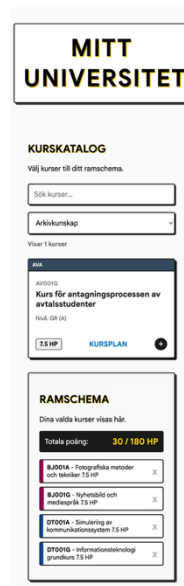
5.1 Gränssnitt och layout

Den slutgiltiga applikationen består av en Single Page Application (SPA) utformad med en responsiv sida-vid-sida-layout (split-screen) för stora skärmar. På skärmens vänstra del visas kurskatalogen och på den högra sidan visas det egna ramschemat (se figur 5.1). Vid användning på mindre skärmar, exempelvis mobiltelefoner, anpassas layouten automatiskt så att ramschemat placeras i botten av vyn (se figur 5.2).

Informationen om kurserna presenteras i ett kort-baserat gränssnitt (Card-based UI). Varje kurskort har färgkodats utifrån sitt ämnesområde enligt en förutbestämd färgkarta, vilket skapar en visuell hierarki. På varje kort finns även en integrerad länk som öppnar den specifika kursplanen i en ny flik (se figur 5.1).



Figur 5.1



Figur 5.2

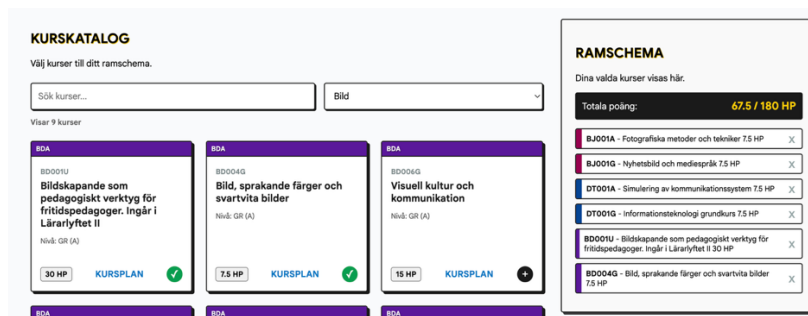
5.2 Sök- och filtreringsfunktion

När applikationen startas renderas inledningsvis inga kurser, vilket ger en ren startsida och säkerställer hög prestanda. Applikationens sök- och filtreringsfunktion reagerar i realtid. När en sökning påbörjas i textfältet, eller när ett ämne väljs i rullgardinsmenyn, filtreras de 4000 kurserna omedelbart och visas på skärmen utan fördröjning. Sökningen hanterar matchningar på både kursnamn och kurskod.

5.3 Ramschema, poängräkning och felhantering

Ramschemat uppdateras i realtid när användaren interagerar med kurskatalogen. När en kurs läggs till adderas dess högskolepoäng automatiskt till den totala poängsumman i schemats övre kant.

Den defensiva kodningen mot dubletter fungerar som tänkt. När en kurs har lagts till i schemat ändras knappen på kurskortet från ett plustecken, till en grön knapp med en bock och knappen inaktiveras för ytterligare klick (se figur 5.3). Detta förhindrar att samma kurs läggs till fler gånger. Skulle applikationen laddas om, eller om webbläsaren stängs ner, finns hela schemat kvar vid nästa besök från samma enhet tack vare lagringen i localStorage.



Figur 5.3

6 Diskussion

Att utveckla den här kurskatalogen har varit både utmanande och väldigt lärorikt. När jag började med projektet kändes det nästan övermäktigt. Det var många saker att tänka på, men tack vare den struktur som fanns i JSON-filen blev det rätt lätt att komma på ett system som kändes stabilt och roligt att använda.

6.1 *Reflektion kring tekniska val*

Att få testa att sätta ihop allt vi har lärt oss i den här TypeScript-kursen, med Angular och speciellt fokus på Signals och Computed-signaler har varit både utmanande och intressant. Det har varit imponerande hur snabbt allt sker på sidan. Att sidan inte behöver laddas om, och att med hjälp av Signals och computed-signaler kunna söka, filtrera, lägga till kurser och räkna ihop poäng, allt i realtid och med väldigt lite kod, har varit kul att lära sig och känns mer lättanvänt och logiskt än vad det verkade i början av kursen.

Det tekniska beslut som jag är mest nöjd med är nog att lät kurslistan starta helt tom, istället för att webbläsaren skulle rendera tusentalskort direkt vid sidladdning, bjuder sidan istället in till interaktion direkt. Det sparar mycket prestanda och gör att appen känns modern.

Det klurigaste jag stötte på var nog att hantera dubbletterna i lärosätets JSON-fil. Det krävde en del felsökning innan jag insåg att jag var tvungen att styra om Angulars change detection i HTML-loopen genom att spåra index-et istället för kurskoderna. Att lösa detta, tillsammans med att synkronisera ramschemat asynkront mot localStorage via en effect(), gav mig större förståelse för den reaktiva programmeringen.

6.2 *Diskussion kring gränssnitt och visuell profil*

När det kom till sidans design valde jag att försöka komma på något som inte bara är en stel tabell, eftersom jag upplever att de är lite tråkiga och inte fungerar så bra på olika skärmstorlekar. Jag inspirerades istället av monopol-kort och tänkte att jag ville använda ett kort-gränssnitt, med hög kontrast, tjocka svarta linjer och hårda skuggor, för att ge en retro-stil med tydlig karaktär och samtidigt behålla en strukturell tydlighet. Jag la till en gul accentfärg i rubrikernas skugga och i poängräknaren, för att lätta upp sidan lite grann.

Färgkodningen via min SCSS-map tycker jag underlättar användarupplevelsen en del. Genom att dela upp de ca 40 ämnesområdena i olika färgfamiljer kan man som student få en överblick över vilka ämnen man har i sitt ramschema utan att behöva läsa texten.

En annan sak som jag tyckte underlättade användandet var att plusknappen som används för att lägga till kursen omvandlas till en grön knapp med en bock som inte går att klicka på, när kursen har lagts till i ramschemat. Denna detalj gjorde

det så mycket tydligare att man hade lagt till en kurs, och inte råkar välja samma igen.

6.3 Framtida utveckling

Om jag hade haft mer tid i projektet hade jag byggt ut valideringen i ramschemat ännu mer. Det hade till exempel varit bra att lägga till en funktion som kontrollerar progressionen i studentens valda kurser, så att systemet kan varna om man försöker lägga till en kurs på B- eller C-nivå utan att först ha lagt till grundkursen på A-nivå inom samma ämne. Sen också att lägga till en export-funktion där man som student kan spara ner sitt ramschema som en PDF-fil.

Sammanfattningsvis känner jag att applikationen har blivit så robust och tillförlitlig som jag kan göra den vid tillfället. Jag har kombinerat de olika sakerna jag har lärt mig under kursens gång, och tycker att jag har fått till ett unikt och användarvänligt yttre som jag är nöjd med.

Källförteckning

- [1] Angular, "Anatomy of a component", Tillgänglig på:
<https://angular.dev/guide/components> Hämtad 2026-05-27
- [2] Angular, "Angular Signals", Tillgänglig på:
<https://angular.dev/guide/signals> Hämtad 2026-05-27
- [3] Sass, "Maps", Tillgänglig på: [https://sass-](https://sass-lang.com/documentation/values/maps/)
[lang.com/documentation/values/maps/](https://sass-lang.com/documentation/values/maps/) Hämtad 2026-05-27
- [4] MDN, "Array.prototype.some()", Tillgänglig på:
[https://developer.mozilla.org/en-](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/some)
[US/docs/Web/JavaScript/Reference/Global_Objects/Array/some](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/some) Hämtad
2026-05-27
- [5] MDN, "Window: localStorage property", Tillgänglig på:
<https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
Hämtad 2026-05-27
- [6] Angular, "Dependency injection in Angular", Tillgänglig på:
<https://angular.dev/guide/di> Hämtad 2026-05-27